SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM071 Name: Shreyas More

Division: A / B Date: _____

GitHub / Portfolio Link (optional): _____

---

1. System Title & Short Description

Tic-Tac-Toe Game-Playing AI (Minimax) — an adversarial search agent that plays Tic-Tac-Toe against a human player on a 3x3 grid. On its turn the agent uses the Minimax algorithm (optionally with Alpha-Beta pruning) to look ahead through all possible future moves, assuming the opponent also plays optimally, and picks the move with the best guaranteed outcome. It solves the general two-player, turn-based, zero-sum game-playing problem, which is a core building block for board-game agents and decision-making under competition.

2. Context Diagram (Level 1)

Figure 1: Context diagram – system boundary and external actors

The **User** plays a move by choosing an empty cell on the board. The **Tic-Tac-Toe AI system** validates the move, replies with its own move, and returns the updated **Game output** — the current board and, at the end, the result (player wins, AI wins, or draw). No external systems are involved; the whole system runs as a self-contained script with no network or API dependency.

3. Container Diagram (Level 2)

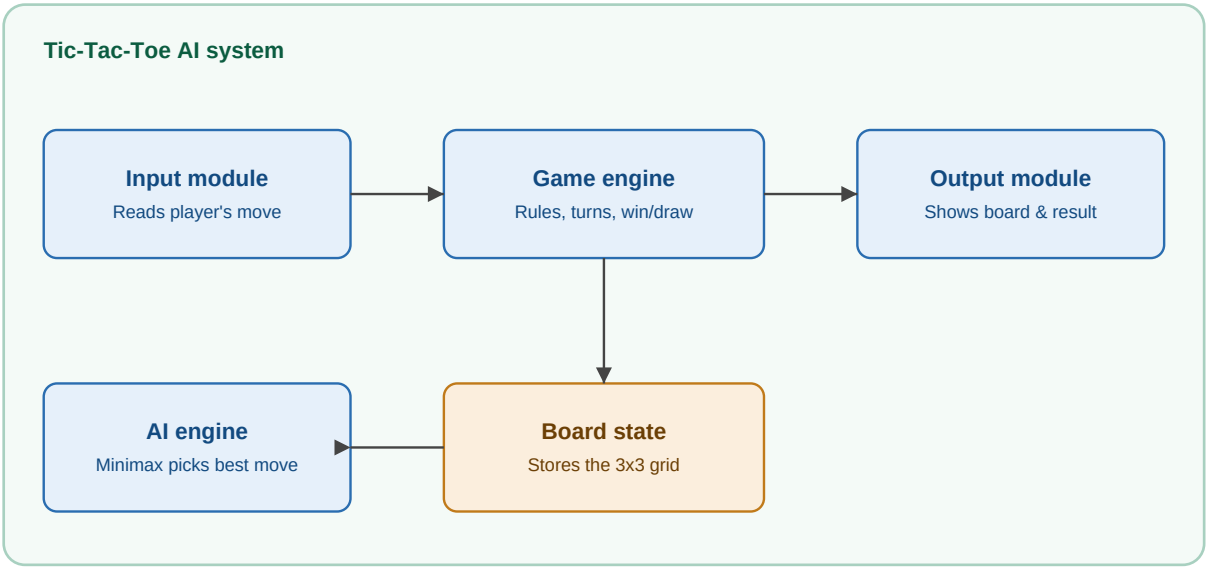

Figure 2: Container diagram – main building blocks inside the system

Explanation of each container:

- **Input module:** Reads the player's chosen cell (1–9) and checks that it is valid and still empty.
- **Game engine:** Controls turn order, applies moves, and checks for a win or a draw after every move.
- **Board state:** Stores the current 3x3 grid (X, O, or empty) shared by the engine and the AI.
- **AI engine:** Runs Minimax over the board state to choose the computer's best move.
- **Output module:** Displays the board after each move and announces the final result.

4. Component Diagram (Level 3)

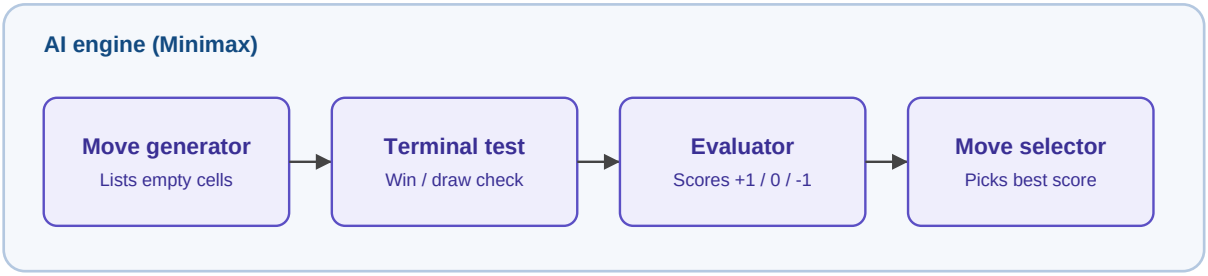

Figure 3: Component diagram – inside the AI engine container

The **AI engine** is the most important container, so it is broken down further. The **Move generator** lists all empty cells the current player can play in. The **Terminal test** checks whether the position is already a win or a draw. The **Evaluator** scores a finished game (+1 for an AI win, 0 for a draw, –1 for a player win). The **Move selector** picks the move with the highest score for the AI (maximizing) while assuming the player always picks the lowest (minimizing).

5. Code Level Overview (Level 4)

Main classes / functions:

- `print_board(board)` : displays the 3x3 grid after every move.
- `check_winner(board)` : returns 'X', 'O', or None by checking all 8 winning lines.
- `is_full(board)` : returns True when no empty cells remain (draw check).
- `minimax(board, depth, is_maximizing)` : recursive search that returns the score of a position.
- `best_move(board)` : tries every empty cell, calls minimax, and returns the cell with the best score.
- `play_game()` : main loop that alternates player and AI turns until the game ends.

6. Design Decisions

The board is stored as a simple list of 9 cells instead of a 2D array or a separate Board class, which keeps move-making and undoing (place, recurse, reset) simple and fast. Minimax is chosen because the game tree of Tic-Tac-Toe is small enough to search completely, so the AI plays perfectly and can never lose. Separating `check_winner` and `minimax` keeps the game rules independent from the AI logic, so the same rules can be reused if the AI is later replaced (for example, by Alpha-Beta pruning or a random player).

7. AI Contribution Note

- **AI tools used (if any):** Claude (Anthropic) — used for code generation, diagram creation, and document formatting.
- **What AI helped with:** Writing/refining the Tic-Tac-Toe and Minimax code; drawing the Context, Container, and Component (C4) diagrams; and formatting this document.
- **What I did myself:** Chose Tic-Tac-Toe as the project, defined the Input / Game Engine / Board State / AI Engine / Output breakdown, decided which container to expand at the Component level, reviewed all AI-generated code and diagrams for correctness, and wrote the design-decision justification.

8. Conclusion

Working through the full C4 model showed how the same system can be described at very different levels of detail without losing consistency — the Context diagram hides the Minimax algorithm completely, while the Code level names the exact functions that implement it. It also made clear that architecture and implementation are separate concerns: the game rules (Game Engine, Board State) are cleanly separated from the decision-making (AI Engine), so the AI strategy can change without touching the rest of the system. Finally, it showed why Minimax suits this problem: the structure stays simple, and the only real complexity lives in one recursive function.